

Movie Ticket Booking App Backend

Documentation

This backend handles the primary functions of a movie ticket booking site. It allows the customers to view the movies, choose the show, reserve a ticket, and facilitate the registration process of the users in a secure environment.

Technological Stack

Node.js: The engine for execution.

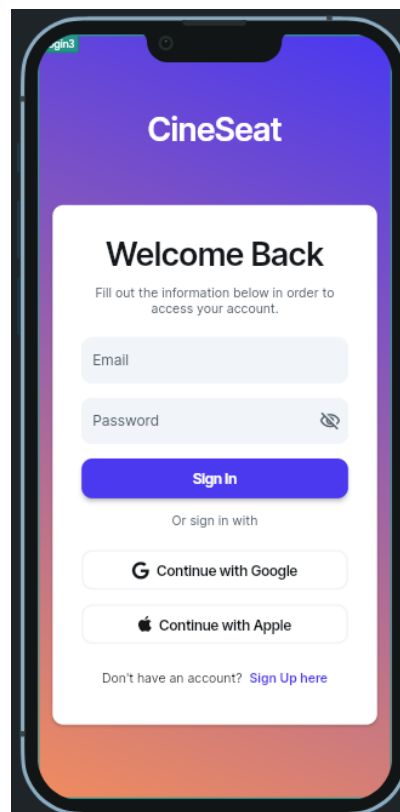
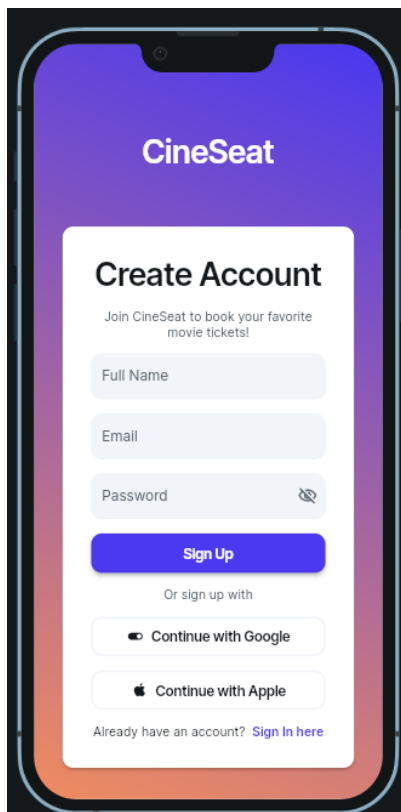
Express.js: The web based framework for APIs.

MongoDB: Mongo based NoSQL cloud database.

Mongoose: The ODM of the database.

JWT (JSON Web Token): For the purpose of authentication.

SignUp Screen:-



Model & Controller of Signup Screen:

username: Distinct name that identifies the user.

email: Electronic mailing address of the user (again unique).

password: This is the hashed value of a user's password which is encrypted using Bcrypt.

Model:-

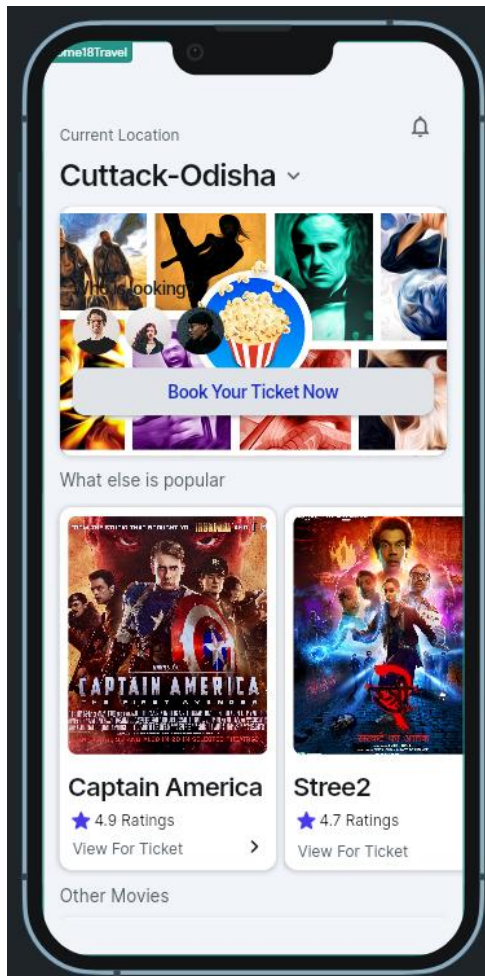
```
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema({
4    username: { type: String, required: true, unique: true },
5    email: { type: String, required: true, unique: true },
6    password: { type: String, required: true },
7  }, { timestamps: true });
8
9  module.exports = mongoose.model('User', userSchema);
```

Controller:

```
1  const User = require('../models/user.model');
2  const bcrypt = require('bcrypt');
3  const jwt = require('jsonwebtoken');
4
5  // Register a new user
6  exports.register = async (req, res) => {
7    const { username, email, password } = req.body;
8    const hashedPassword = await bcrypt.hash(password, 10);
9    const newUser = new User({ username, email, password: hashedPassword });
10
11    try {
12      const savedUser = await newUser.save();
13      res.status(201).json(savedUser);
14    } catch (error) {
15      res.status(400).json({ message: error.message });
16    }
17  };
18
19  // User login
20  exports.login = async (req, res) => {
21    const { username, password } = req.body;
22    const user = await User.findOne({ username });
23    if (!user) return res.status(404).json({ message: 'User not found' });
24
25    const isMatch = await bcrypt.compare(password, user.password);
26    if (!isMatch) return res.status(401).json({ message: 'Invalid credentials' });
27
28    const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET);
29    res.json({ token });
30  };
```

Home Page:

It is showing all the movie list.



Model & Controller of the Home Page:-

title: Title of the film.

description: An outline of the plot or story in a few sentences.

genre: Film category or type - Action, Comedy etc.

duration: Time that the film takes in minutes.

releaseDate: The date when the film is out or released.

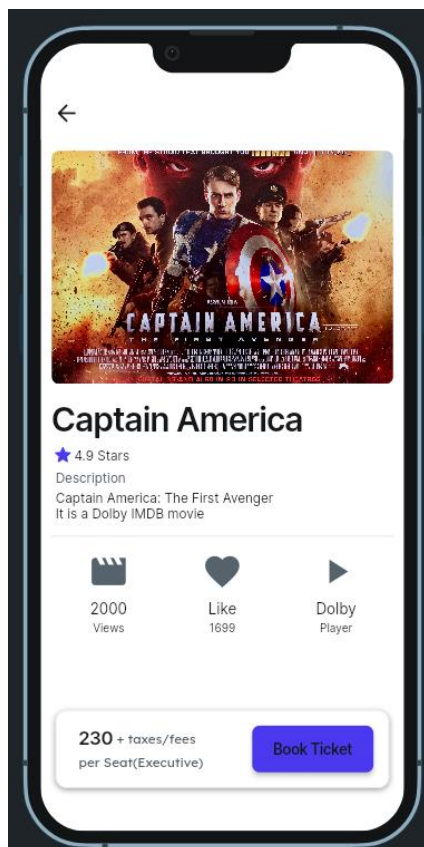
Model:-

```
1  const mongoose = require('mongoose');
2
3  const movieSchema = new mongoose.Schema({
4    title: { type: String, required: true },
5    description: { type: String },
6    genre: { type: String, required: true },
7    duration: { type: Number, required: true },
8    releaseDate: { type: Date, required: true },
9    language: { type: String, required: true }
10  }, { timestamps: true });
11
12  module.exports = mongoose.model('Movie', movieSchema);
```

Controller:-

```
1  const Movie = require('../models/movie.model');
2
3  exports.createMovie = async (req, res) => {
4    const newMovie = new Movie(req.body);
5
6    try {
7      const savedMovie = await newMovie.save();
8      res.status(201).json(savedMovie);
9    } catch (error) {
10     res.status(400).json({ message: error.message });
11   }
12 };
13
14 exports.getMovies = async (req, res) => {
15   const movies = await Movie.find();
16   res.json(movies);
17 };
```

Movie Description Page



Model & Controller of the Movie Description page:

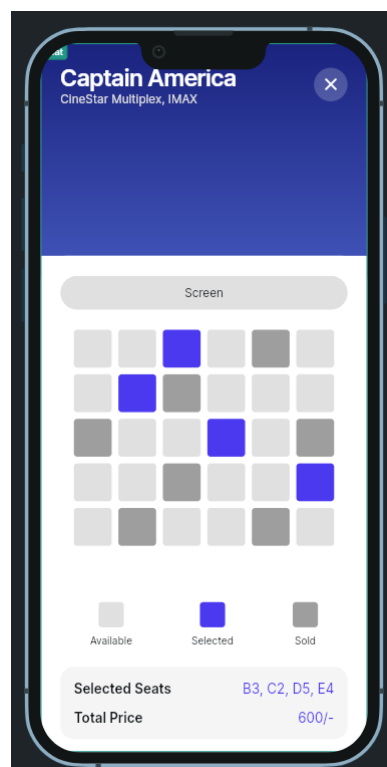
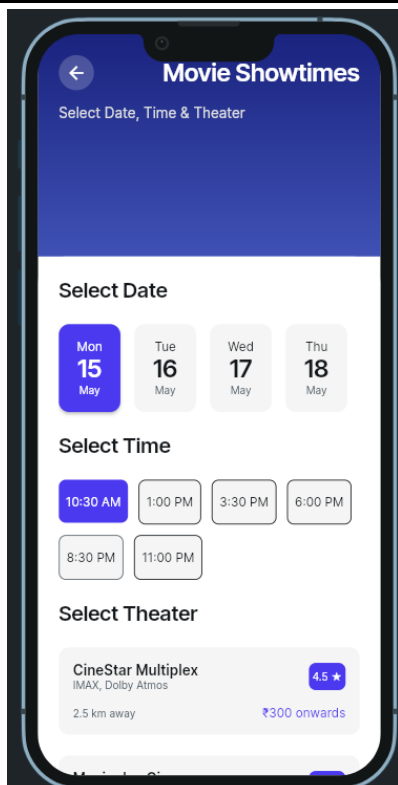
Model:

```
1  const mongoose = require('mongoose');
2
3  const movieDescription = new mongoose.Schema({
4    title: { type: String, required: true },
5    description: { type: String },
6    genre: { type: String, required: true },
7    duration: { type: Number, required: true },
8    releaseDate: { type: Date, required: true },
9    language: { type: String, required: true },
10   likes: { type: Number, default: 0 },
11   views: { type: Number, default: 0 }
12 }, { timestamps: true });
13
14 module.exports = mongoose.model('Movie', movieDescription);
```

Controller:

```
1  const Movie = require('../models/movie.model');
2  const Showtime = require('../models/showtime.model');
3
4  exports.getMovieWithShowtimes = async (req, res) => {
5    const movieId = req.params.movieId;
6
7    try {
8      const movie = await Movie.findById(movieId);
9      if (!movie) return res.status(404).json({ message: 'Movie not found' });
10
11      movie.views += 1;
12      await movie.save();
13
14      const showtimes = await Showtime.find({ movie: movieId });
15      res.json({
16        movie: {
17          title: movie.title,
18          description: movie.description,
19          genre: movie.genre,
20          duration: movie.duration,
21          releaseDate: movie.releaseDate,
22          language: movie.language,
23          likes: movie.likes,
24          views: movie.views
25        },
26        showtimes: showtimes.map(showtime => ({
27          theater: showtime.theater,
28          date: showtime.date,
29          time: showtime.time,
30          availableSeats: showtime.availableSeats,
31          ticketPrice: showtime.ticketPrice
32        })))
33      });
34    } catch (error) {
35      res.status(500).json({ message: error.message });
36    }
37  };
```

Seat/Date/Theater Selection Page:



Model & Controller of Seat Selection Page:

Model:

```
1  const mongoose = require('mongoose');
2
3  const seatSchema = new mongoose.Schema({
4    seatNumber: { type: String, required: true },
5    isBooked: { type: Boolean, default: false },
6  });
7
8  const showtimeSchema = new mongoose.Schema({
9    movie: { type: mongoose.Schema.Types.ObjectId, ref: 'Movie', required: true },
10   date: { type: Date, required: true },
11   time: { type: String, required: true },
12   availableSeats: [seatSchema],
13   ticketPrice: { type: Number, required: true }
14 }, { timestamps: true });
15
16 module.exports = mongoose.model('Showtime', showtimeSchema);
```

```
1  const mongoose = require('mongoose');
2
3  const bookingSchema = new mongoose.Schema({
4    user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
5    showtime: { type: mongoose.Schema.Types.ObjectId, ref: 'Showtime', required: true },
6    selectedSeats: [{ type: String, required: true }],
7    totalPrice: { type: Number, required: true },
8  }, { timestamps: true });
9
10 module.exports = mongoose.model('Booking', bookingSchema);
```

Controller:

```
const Movie = require('../models/movie.model');
const Showtime = require('../models/showtime.model');
const Booking = require('../models/booking.model');

exports.getShowtimeDetails = async (req, res) => {
  const showtimeId = req.params.showtimeId;

  try {
    const showtime = await Showtime.findById(showtimeId).populate('movie');
    if (!showtime) return res.status(404).json({ message: 'Showtime not found' });

    res.json({
      movie: showtime.movie,
      date: showtime.date,
      time: showtime.time,
      availableSeats: showtime.availableSeats,
      ticketPrice: showtime.ticketPrice
    });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Once the user selects the seats and confirms, the controller will:

Mark the seats as booked.

Save the booking.

```
24 exports.bookSeats = async (req, res) => {
25   const { showtimeId, userId, selectedSeats } = req.body;
26
27   try {
28     const showtime = await Showtime.findById(showtimeId);
29     if (!showtime) return res.status(404).json({ message: 'Showtime not found' });
30
31     const unavailableSeats = selectedSeats.filter(seat =>
32       showtime.availableSeats.find(s => s.seatNumber === seat && s.isBooked)
33     );
34     if (unavailableSeats.length > 0) {
35       return res.status(400).json({ message: `Seats ${unavailableSeats.join(', ')} are already booked` });
36     }
37
38     selectedSeats.forEach(seat => {
39       const seatToBook = showtime.availableSeats.find(s => s.seatNumber === seat);
40       seatToBook.isBooked = true;
41     });
42
43     await showtime.save();
44
45     const totalPrice = selectedSeats.length * showtime.ticketPrice;
46
47     const booking = new Booking({
48       user: userId,
49       showtime: showtimeId,
50       selectedSeats: selectedSeats,
51       totalPrice: totalPrice
52     });
53
54     const savedBooking = await booking.save();
55
56     res.status(201).json({ message: 'Booking confirmed', booking: savedBooking });
57   } catch (error) {
58     res.status(500).json({ message: error.message });
59   }
60 };
```

Route Setup for Seat Selection Page

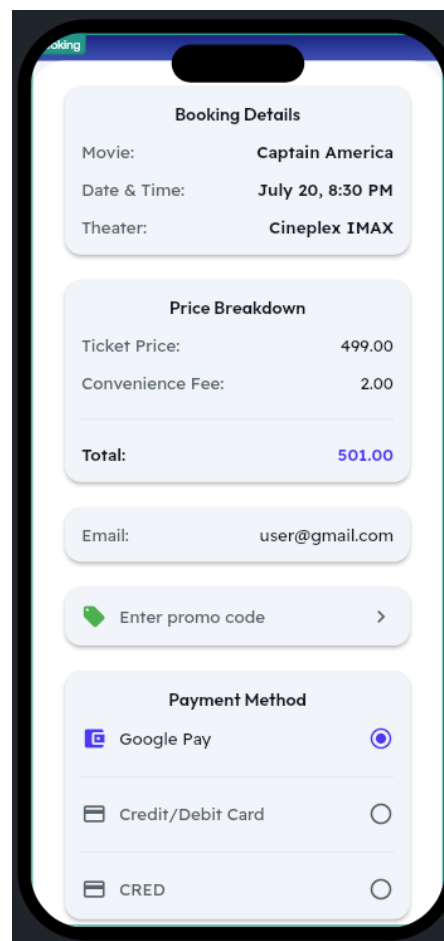
```
const express = require('express');
const router = express.Router();
const showtimeController = require('../controllers/showtime.controller');
const bookingController = require('../controllers/booking.controller');

router.get('/showtimes/:showtimeId', showtimeController.getShowtimeDetails);

router.post('/bookings', bookingController.bookSeats);

module.exports = router;
```


Booking Details Page with Payment



Models & Controller of Booking Details Page

Model:

movie: Mentioning the movie title from which the booking is being made .

showtime: Including all information about the date, time and theater for the particular show.

userEmail: User's email address used for communication.

selectedSeats: A collection of seats that the user has booked.

ticketPrice: Overall cost for the booked seats.

convenienceFee: Extra money charged for services.

totalPrice: Adjusted amount after straightforward added convenience fee and discounts applied if any.

promoCode: A code used for a discount by a user if available.

paymentStatus: track the payments occasional status, whether it's pending, completed, or failed.

paymentMethod: Mode of payment selected by the user.

```
1  const mongoose = require('mongoose');
2
3  const bookingSchema = new mongoose.Schema({
4    movie: { type: mongoose.Schema.Types.ObjectId, ref: 'Movie', required: true },
5    showtime: { type: mongoose.Schema.Types.ObjectId, ref: 'Showtime', required: true },
6    userEmail: { type: String, required: true },
7    selectedSeats: [{ type: String, required: true }],
8    ticketPrice: { type: Number, required: true },
9    convenienceFee: { type: Number, default: 0 },
10   totalPrice: { type: Number, required: true },
11   promoCode: { type: String },
12   discountAmount: { type: Number, default: 0 },
13   paymentStatus: { type: String, enum: ['pending', 'completed', 'failed'], default: 'pending' },
14   paymentMethod: { type: String, enum: ['Google Pay', 'Credit/Debit Card', 'CRED'], required: true },
15 }, { timestamps: true });
16
17 module.exports = mongoose.model('Booking', bookingSchema);
```

Controller:-

```
demo 7 ...
1  exports.applyPromoCode = async (req, res) => {
2    const { promoCode } = req.body;
3
4    const validPromoCode = 'DISCOUNT10';
5    const discountAmount = 10;
6
7    if (promoCode === validPromoCode) {
8      return res.json({ isValid: true, discountAmount });
9    }
10
11    res.status(400).json({ isValid: false, message: 'Invalid promo code' });
12  };
```

```

1  const Booking = require('../models/booking.model');
2
3  exports.processBooking = async (req, res) => {
4    const { showtimeId, userEmail, selectedSeats, promoCode, paymentMethod } = req.body;
5    try {
6      const showtime = await Showtime.findById(showtimeId).populate('movie');
7      if (!showtime) return res.status(404).json({ message: 'Showtime not found' });
8      const ticketPrice = selectedSeats.length * showtime.ticketPrice;
9      const convenienceFee = 2;
10     let totalPrice = ticketPrice + convenienceFee;
11     let discountAmount = 0;
12
13     if (promoCode) {
14       const promoResponse = await axios.post('/apply-promo', { promoCode });
15       if (promoResponse.data.isValid) {
16         discountAmount = promoResponse.data.discountAmount;
17         totalPrice -= discountAmount;
18       }
19     }
20     // Create a booking record
21     const booking = new Booking({
22       movie: showtime.movie,
23       showtime: showtimeId,
24       userEmail,
25       selectedSeats,
26       ticketPrice,
27       convenienceFee,
28       totalPrice,
29       promoCode,
30       discountAmount,
31       paymentMethod,
32       paymentStatus: 'pending'
33     });
34     const savedBooking = await booking.save();
35     res.status(201).json({ message: 'Booking created', booking: savedBooking });
36   } catch (error) {
37     res.status(500).json({ message: error.message });

```

```

41  exports.initiatePayment = async (req, res) => {
42    const { bookingId, paymentMethod } = req.body;
43
44    try {
45      const booking = await Booking.findById(bookingId);
46      if (!booking) return res.status(404).json({ message: 'Booking not found' });
47
48      const paymentSuccess = true;
49
50      if (paymentSuccess) {
51        booking.paymentStatus = 'completed';
52        await booking.save();
53        res.status(200).json({ message: 'Payment successful', booking });
54      } else {
55        booking.paymentStatus = 'failed';
56        await booking.save();
57        res.status(400).json({ message: 'Payment failed', booking });
58      }
59    } catch (error) {
60      res.status(500).json({ message: error.message });
61    }
62  };

```

Route Setup

```
const express = require('express');
const router = express.Router();
const bookingController = require('../controllers/booking.controller');

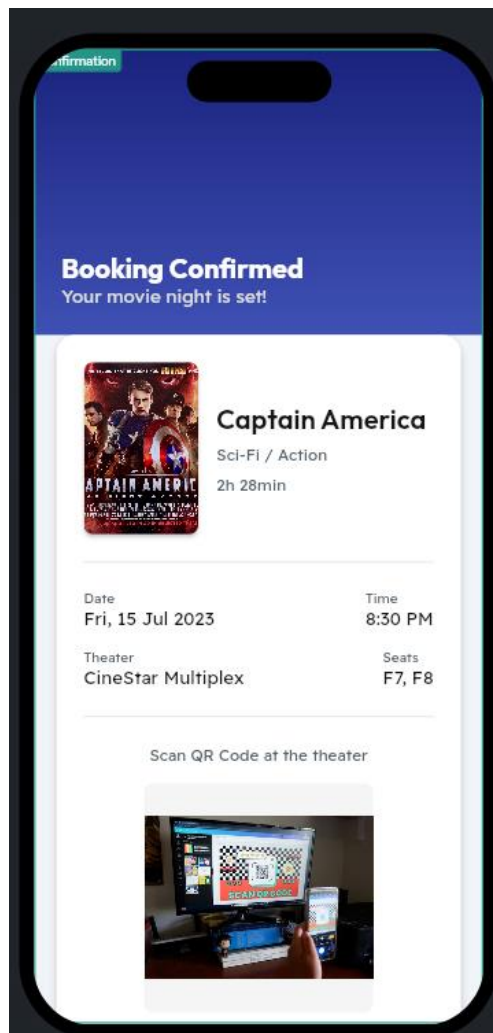
router.post('/bookings', bookingController.processBooking);

router.post('/apply-promo', bookingController.applyPromoCode);

router.post('/payments', bookingController.initiatePayment);

module.exports = router;
```

Booking Confirmation Page:



Model & Controller of Booking Confirmation Page:

qrCodeUrl: A field designated to hold the address or information contained in the QR code.

bookingStatus: A field which indicates the state of the booking, for instance after the booking is done, it is changed to confirmed.

Model:

```
1  const mongoose = require('mongoose');
2
3  const bookingSchema = new mongoose.Schema({
4    movie: { type: mongoose.Schema.Types.ObjectId, ref: 'Movie', required: true },
5    showtime: { type: mongoose.Schema.Types.ObjectId, ref: 'Showtime', required: true },
6    userEmail: { type: String, required: true },
7    selectedSeats: [{ type: String, required: true }],
8    ticketPrice: { type: Number, required: true },
9    convenienceFee: { type: Number, default: 0 },
10   totalPrice: { type: Number, required: true },
11   promoCode: { type: String },
12   discountAmount: { type: Number, default: 0 },
13   paymentStatus: { type: String, enum: ['pending', 'completed', 'failed'], default: 'pending' },
14   paymentMethod: { type: String, enum: ['Google Pay', 'Credit/Debit Card', 'CRED'], required: true },
15   qrCodeUrl: { type: String },
16   bookingStatus: { type: String, enum: ['confirmed', 'pending', 'canceled'], default: 'pending' },
17 }, { timestamps: true });
18
19 module.exports = mongoose.model('Booking', bookingSchema);
```

Controller:

```
1  const QRCode = require('qrcode');
2  const Booking = require('../models/booking.model');
3
4  // Generate booking confirmation and QR code
5  exports.confirmBooking = async (req, res) => {
6    const { bookingId } = req.params;
7
8    try {
9      const booking = await Booking.findById(bookingId).populate('movie showtime');
10     if (!booking) return res.status(404).json({ message: 'Booking not found' });
11
12     const qrCodeData = `https://www.booking.com/${booking._id}`;
13     const qrCodeUrl = await QRCode.toDataURL(qrCodeData);
14
15     booking.qrCodeUrl = qrCodeUrl;
16     booking.bookingStatus = 'confirmed';
17     await booking.save();
18
19     res.status(200).json({
20       message: 'Booking confirmed',
21       booking: {
22         movie: booking.movie,
23         showtime: booking.showtime,
24         selectedSeats: booking.selectedSeats,
25         qrCodeUrl: booking.qrCodeUrl,
26         bookingStatus: booking.bookingStatus,
27       }
28     });
29   } catch (error) {
30     res.status(500).json({ message: error.message });
31   }
32 };
```

Booking Confirmation Controller

```
34 exports.getBookingConfirmation = async (req, res) => {
35   const { bookingId } = req.params;
36
37   try {
38     const booking = await Booking.findById(bookingId).populate('movie showtime');
39     if (!booking) return res.status(404).json({ message: 'Booking not found' });
40
41     res.status(200).json({
42       movie: booking.movie,
43       showtime: booking.showtime,
44       selectedSeats: booking.selectedSeats,
45       qrCodeUrl: booking.qrCodeUrl,
46       bookingStatus: booking.bookingStatus,
47     });
48   } catch (error) {
49     res.status(500).json({ message: error.message });
50   }
51 };
```

Routes

```
const express = require('express');
const router = express.Router();
const bookingController = require('../controllers/booking.controller');

router.get('/bookings/:bookingId/confirm', bookingController.confirmBooking);

router.get('/bookings/:bookingId/confirmation', bookingController.getBookingConfirmation);

module.exports = router;
```

Conclusion:

The movie ticket booking app offers a seamless and user-friendly experience for purchasing tickets, selecting seats, and making payments. It integrates several core functionalities, from searching for movies to completing a booking with a QR code for entry.